Week 2 - Amazon EKS basics for CI/CD (light)

DOP-C02 Domain 1 (SDLC Automation). You don't need to be a Kubernetes expert. The exam tests how a pipeline **authenticates to and deploys into** EKS, how pods get **AWS permissions**, which **deployment strategies** apply (and that **CodeDeploy doesn't support EKS**), and basic logging and monitoring.

Part 1 - Building blocks

What EKS manages and what you manage

Layer	Who runs it	Notes
Control plane (API server, etcd, scheduler)	**AWS**	Multi-AZ, highly available. You pay per cluster per hour
Data plane (where pods run)	You choose	See the options below

Data plane option	What it is	Use when
Managed node groups	EC2 Auto Scaling groups that EKS creates and updates (AMI updates, graceful draining)	Default choice for EC2 nodes
Self-managed nodes	Your own ASG with the EKS AMI	Full control, custom AMIs, more work
Fargate profiles	Pods matching a **namespace and labels** run serverless on Fargate, one pod per micro-VM	No nodes to manage. **No DaemonSets, no privileged pods, no EBS volumes**
EKS Auto Mode	AWS manages nodes, scaling, load balancer controller and storage drivers	Least operational work (newer)
Karpenter	Open-source autoscaler that launches right-sized EC2 nodes quickly	Faster, cheaper node scaling than Cluster Autoscaler

Kubernetes objects to recognise

Object	Purpose
Pod	One or more containers scheduled together. The smallest unit
Deployment	Keeps N replicas running; **rolling updates and rollbacks** through ReplicaSets
Service	Stable endpoint for pods: `ClusterIP` (internal), `NodePort`, `LoadBalancer` (an **NLB** through the AWS Load Balancer Controller)
Ingress	HTTP routing, which becomes an **ALB** through the **AWS Load Balancer Controller**
ConfigMap / Secret	Configuration and secrets. Secrets are only base64 encoded: use **KMS envelope encryption** or **Secrets Manager** with the Secrets Store CSI driver
Namespace	Logical separation per team or environment
ServiceAccount	The identity a pod runs as, linked to an IAM role (Part 3)
HorizontalPodAutoscaler	Scales pods on CPU, memory or custom metrics

Add-ons and versions

- **Managed add-ons:** VPC CNI (pods get VPC IPs), CoreDNS, kube-proxy, EBS/EFS CSI drivers, Pod Identity Agent, CloudWatch Observability agent.
- **Upgrades:** control plane **one minor version at a time**, then add-ons and nodes. Check deprecated APIs in manifests first.
- **Support:** standard support about **14 months** per version; **extended support** about 12 more months at **extra cost**.

- **Blue/green cluster upgrade:** a new cluster on the new version, deploy to it, shift traffic with Route 53. The low-risk answer for big version jumps.

Part 2 - How a pipeline deploys to EKS

The classic pipeline

```
CodeCommit / GitHub -> CodeBuild (build image) -> ECR -> CodeBuild (kubectl / helm) -> EKS
                                                          or CodePipeline "Amazon EKS" deploy action
```

1. **Build:** CodeBuild builds the image (**privileged mode** for Docker) and pushes it to ECR with a unique tag (commit SHA).
2. **Update the manifest:** `kubectl set image`, `kustomize edit set image`, or `helm upgrade --set image.tag=...`.
3. **Deploy:** `kubectl apply -f k8s/` or `helm upgrade --install`, then `kubectl rollout status` so the build fails if the rollout fails.

Option	Notes
CodeBuild running kubectl or Helm	The classic exam answer. Flexible
CodePipeline Amazon EKS deploy action	Newer (2025). Deploys **kubectl manifests or Helm charts** directly, including private clusters through VPC settings
GitOps (Argo CD / Flux)	**Pull model**: a controller in the cluster watches Git and applies changes. The pipeline only updates Git, needs **no cluster credentials**, and drift is fixed automatically

- **CodeDeploy does NOT support EKS.** It supports EC2/on-premises, Lambda and ECS only.

buildspec example

```
version: 0.2
env:
  variables:
    CLUSTER: prod-cluster
phases:
  pre_build:
    commands:
      - aws ecr get-login-password | docker login --username AWS --password-stdin $REPO_HOST
      - TAG=$CODEBUILD_RESOLVED_SOURCE_VERSION
  build:
    commands:
      - docker build -t $REPO_URI:$TAG .
      - docker push $REPO_URI:$TAG
  post_build:
    commands:
      - aws eks update-kubeconfig --name $CLUSTER --region $AWS_REGION
      - kubectl set image deployment/web web=$REPO_URI:$TAG -n shop
      - kubectl rollout status deployment/web -n shop --timeout=300s
```

Authenticating the pipeline to the cluster (most tested)

Two layers must **both** allow the pipeline:

1. **IAM:** the CodeBuild role needs `eks:DescribeCluster`, which `aws eks update-kubeconfig` calls to get the endpoint and certificate.
2. **Kubernetes RBAC:** the role must be **mapped into the cluster**.

Method	How	Notes
Access entries (recommended)	`create-access-entry` for the role ARN plus `associate-access-policy`, for example `AmazonEKSClusterAdminPolicy` or `AmazonEKSEditPolicy` **scoped to a namespace**	Managed through the EKS API, CloudFormation or Terraform. No ConfigMap editing
`aws-auth` **ConfigMap** (legacy)	Add the role under `mapRoles` with Kubernetes groups	Easy to break; a bad edit can lock everyone out

```
aws eks create-access-entry --cluster-name prod-cluster \
  --principal-arn arn:aws:iam::111122223333:role/codebuild-deploy-role
aws eks associate-access-policy --cluster-name prod-cluster \
  --principal-arn arn:aws:iam::111122223333:role/codebuild-deploy-role \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSEditPolicy \
  --access-scope type=namespace,namespaces=shop
```

- By default **only the identity that created the cluster** has admin access. Others get `You must be logged in to the server (Unauthorized)` until mapped.
- **Private API endpoint:** CodeBuild must run **inside the VPC** with subnets and security groups that reach the endpoint.
- **Least privilege:** give the pipeline role only the namespaces it deploys to.

Part 3 - Pods calling AWS services

Method	How	Exam view
EKS Pod Identity (newer)	**Pod Identity Agent** add-on plus a **pod identity association** (cluster, namespace, service account, IAM role). The role trusts `pods.eks.amazonaws.com`	Simplest. No OIDC setup; roles reusable across clusters
IRSA (IAM Roles for Service Accounts)	An **IAM OIDC provider** for the cluster; the role trusts it with conditions on the service account; annotate the service account with `eks.amazonaws.com/role-arn`	Well known, still a common exam answer
Node instance role	Every pod on the node gets the node's permissions	**Avoid**: not least privilege

- **ECR image pulls:** EC2 nodes use the **node role** (`AmazonEC2ContainerRegistryReadOnly`). Fargate pods use the **pod execution role**.
- Pods in different namespaces needing different permissions means **IRSA or Pod Identity per service account**.

Part 4 - Deployment strategies on EKS

Rolling update (default for a Deployment)

```
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%        # extra pods allowed above replicas
      maxUnavailable: 0    # keep full capacity
  template:
    spec:
      containers:
        - name: web
          image: 111122223333.dkr.ecr.us-east-1.amazonaws.com/web:abc123
          readinessProbe:            # only send traffic when ready
            httpGet: { path: /health, port: 8080 }
          livenessProbe:             # restart if stuck
            httpGet: { path: /live, port: 8080 }
```

- `maxSurge` / `maxUnavailable` do the same job as ECS `maximumPercent` / `minimumHealthyPercent`.
- **Readiness probe:** no traffic until the pod is ready. Without it, traffic reaches starting pods and users get errors during deployments.
- **Liveness probe: restarts** a stuck container. Too strict a probe causes restart loops.
- `kubectl rollout status` waits; `kubectl rollout undo` rolls back to the previous ReplicaSet; `kubectl rollout history` lists revisions.
- `Recreate` strategy kills all pods, then starts new ones: downtime.
- **Helm:** `helm upgrade --install --atomic --wait` **rolls back automatically** if the release fails; `helm rollback <release> <revision>` manually.

Blue/green and canary on EKS

Pattern	How on EKS
Blue/green	Two Deployments (`web-blue`, `web-green`); switch the **Service selector** (`version: blue` to `green`) or the Ingress target
Canary (simple)	A few canary pods behind the same Service; traffic share follows the pod ratio
Canary (weighted)	**ALB Ingress weighted target groups** (AWS Load Balancer Controller annotations) or a service mesh
Progressive delivery	**Argo Rollouts** or **Flagger**: canary or blue/green steps, metric analysis (CloudWatch, Prometheus), **automatic rollback**
Whole-cluster blue/green	New cluster plus Route 53 weighted records, for Kubernetes version upgrades

Part 5 - Observability and security

- **Control plane logs** (`api`, `audit`, `authenticator`, `controllerManager`, `scheduler`) go to **CloudWatch Logs**. **Off by default**; turn on per type. Who deleted a Deployment? The **audit** log.
- **Container Insights** for EKS through the **CloudWatch Observability add-on**; **Fluent Bit** ships container logs.
- **Amazon Managed Service for Prometheus** and **Amazon Managed Grafana** for Prometheus-style metrics.
- **AWS Load Balancer Controller** creates ALBs and NLBs from Ingress and Service objects. It needs IAM through IRSA or Pod Identity. Fargate pods need target type `ip`.
- **GuardDuty EKS Protection** for audit log and runtime threats. **ECR scanning** (Inspector) before deployment. **Kyverno or OPA Gatekeeper** to block unsigned or unscanned images.
- **Scaling: HPA** for pods (needs **metrics-server**); **Cluster Autoscaler** or **Karpenter** for nodes.

Part 6 - Exam traps

Trap	Truth
Use CodeDeploy blue/green to deploy to EKS	**CodeDeploy doesn't support EKS.** Use kubectl or Helm from CodeBuild, the CodePipeline EKS action, GitOps, or Argo Rollouts / Flagger
CodeBuild gets **Unauthorized** from kubectl	The role isn't mapped in the cluster. Add an **access entry** (or `aws-auth` mapping). IAM alone is not enough
`update-kubeconfig` fails with AccessDenied	The IAM role lacks `eks:DescribeCluster`
CodeBuild times out connecting to the cluster	**Private endpoint** not reachable. Run CodeBuild **in the VPC**
Give pods S3 access with the node role	Too broad. Use **IRSA or EKS Pod Identity**
Errors during every rolling update	Add a **readinessProbe** and set `maxUnavailable: 0`
Pods keep restarting	A wrong **livenessProbe** (delay or timeout too short), or OOMKilled by memory limits
Kubernetes Secrets are encrypted, so passwords are safe there	Only base64. Use **KMS envelope encryption** or **Secrets Manager** with the CSI driver
Run a DaemonSet on Fargate	Not supported. Use EC2 nodes or sidecars
Who changed this in the cluster?	**Control plane audit logs**. CloudTrail shows EKS API calls, not Kubernetes API calls
Pipeline must deploy without cluster credentials, drift must self-heal	**GitOps** (Argo CD / Flux)
Skip from 1.29 to 1.31 in one step	Not in place. **One minor version at a time**, or a **new cluster with blue/green**
Pipeline should fail if the deployment isn't healthy	`kubectl rollout status --timeout`, or `helm --atomic --wait`

Part 7 - Hands-on lab (about 60 to 90 minutes)

- **Cost warning:** the control plane costs about **$0.10 per hour**, plus nodes and NAT. **Delete everything at the end.**

Lab 1 - Create a cluster and deploy

```
# 1. Small cluster (eksctl creates the VPC, node group and kubeconfig)
eksctl create cluster --name lab --region us-east-1 --nodes 2 --node-type t3.medium

# 2. Deploy nginx with 3 replicas and expose it
kubectl create deployment web --image=public.ecr.aws/nginx/nginx:1.25 --replicas=3
kubectl expose deployment web --port=80 --type=LoadBalancer
kubectl get svc web -w            # wait for the hostname, then open it
```

Lab 2 - Rolling update and rollback

```
kubectl set image deployment/web nginx=public.ecr.aws/nginx/nginx:1.27
kubectl rollout status deployment/web
kubectl rollout history deployment/web

# Break it on purpose, then watch and roll back
kubectl set image deployment/web nginx=public.ecr.aws/nginx/nginx:does-not-exist
kubectl get pods                  # ImagePullBackOff, old pods keep serving
kubectl rollout undo deployment/web
```

Lab 3 - Let a CodeBuild role deploy

1. Create an IAM role for CodeBuild with `eks:DescribeCluster`.
2. Run `kubectl get pods` from CodeBuild **before** mapping the role: you see **Unauthorized**.
3. Add an **access entry** with `AmazonEKSEditPolicy` scoped to the `default` namespace.
4. Run the buildspec from Part 2. It deploys and waits with `rollout status`.

Lab 4 - Pod permissions with Pod Identity

1. Install the **Pod Identity Agent** add-on.
2. Create an IAM role trusting `pods.eks.amazonaws.com` with read-only S3 access.
3. `eksctl create podidentityassociation --cluster lab --namespace default --service-account-name s3-reader --role-arn <role>`
4. Run a pod with that service account and `aws s3 ls`: it works. Without the association it is denied.

Lab 5 - Logging

1. Turn on **audit** control plane logging in the EKS console.
2. Delete a pod.
3. Find the delete event in CloudWatch Logs Insights.

Clean up

```
kubectl delete svc web            # removes the load balancer first
eksctl delete cluster --name lab --region us-east-1
```

Part 8 - Check yourself

Questions

1. A CodeBuild project runs `kubectl apply` and gets "You must be logged in to the server (Unauthorized)". IAM allows `eks:DescribeCluster`. What is missing?
2. A team wants CodeDeploy canary deployments for its EKS services. What do you tell them?
3. Pods in the `billing` namespace need DynamoDB access and other pods must not have it. What should you use?
4. Users see 502 errors for a few seconds during every rolling update. Which setting is most likely missing?
5. The pipeline must fail and roll back automatically if a Helm release doesn't become healthy. Which flags?
6. Security wants to know who deleted a Deployment in the cluster. Where do you look?
7. The cluster API endpoint is private and CodeBuild can't reach it. What is the fix?
8. The pipeline must not hold cluster credentials, and manual changes must be reverted automatically. Which approach?
9. Which data plane option needs no node management, and what are two things it can't run?
10. A cluster on 1.29 must go to 1.31 with minimal risk. What are the options?

Answers

1. The role isn't **mapped into the cluster**. Add an **EKS access entry** with an access policy (or an `aws-auth` mapping).
2. **CodeDeploy doesn't support EKS.** Use Argo Rollouts or Flagger, ALB Ingress weights, or kubectl/Helm from CodeBuild.
3. **IRSA or EKS Pod Identity**: an IAM role tied to the billing service account, not the node role.
4. A **readinessProbe**, plus `maxUnavailable: 0` with enough `maxSurge`.
5. `helm upgrade --install --atomic --wait` (with a `--timeout`).
6. **EKS control plane audit logs** in CloudWatch Logs (must be turned on).
7. Run **CodeBuild in the VPC** with network access to the endpoint, or enable the public endpoint restricted to allowed CIDRs.
8. **GitOps** (Argo CD or Flux): pull model with automatic drift correction.
9. **Fargate profiles**: no **DaemonSets**, no **privileged pods**, no EBS volumes (EFS works). EKS Auto Mode is also low management but runs EC2 nodes for you.
10. In place **one minor version at a time** (1.29 to 1.30 to 1.31; control plane first, then add-ons and nodes), or a **new 1.31 cluster** with traffic shifted by **Route 53 (blue/green)**.

Quick memory summary

- AWS runs the **control plane**; you choose managed node groups, self-managed nodes, **Fargate profiles** (no DaemonSets, no privileged pods) or **Auto Mode**.
- Pipeline: CodeBuild builds and pushes to ECR, then **kubectl / Helm** (or the CodePipeline **EKS action**, or **GitOps**). **No CodeDeploy for EKS.**
- Pipeline auth = `eks:DescribeCluster` (IAM) **plus an access entry** (or `aws-auth`) for Kubernetes RBAC. Private endpoint means CodeBuild in the VPC.
- Pod permissions = **Pod Identity** or **IRSA**, never the node role.
- Rolling: `maxSurge`, `maxUnavailable`, **readinessProbe**; `rollout status` / `rollout undo`; `helm --atomic --wait`.
- Canary / blue-green: Service selector switch, ALB Ingress weights, **Argo Rollouts / Flagger**; cluster upgrades with **new cluster + Route 53**.
- Logs: control plane logs (**audit**) are off by default; Container Insights add-on; GuardDuty EKS Protection.